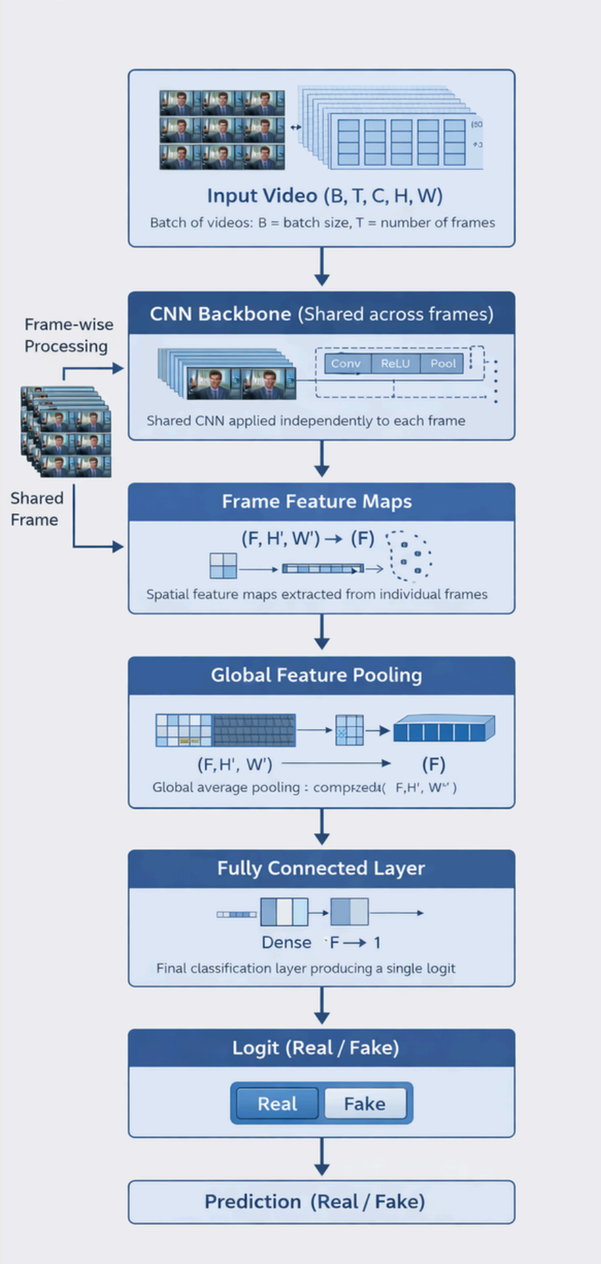
Deep fake Detection

Only CNN Model



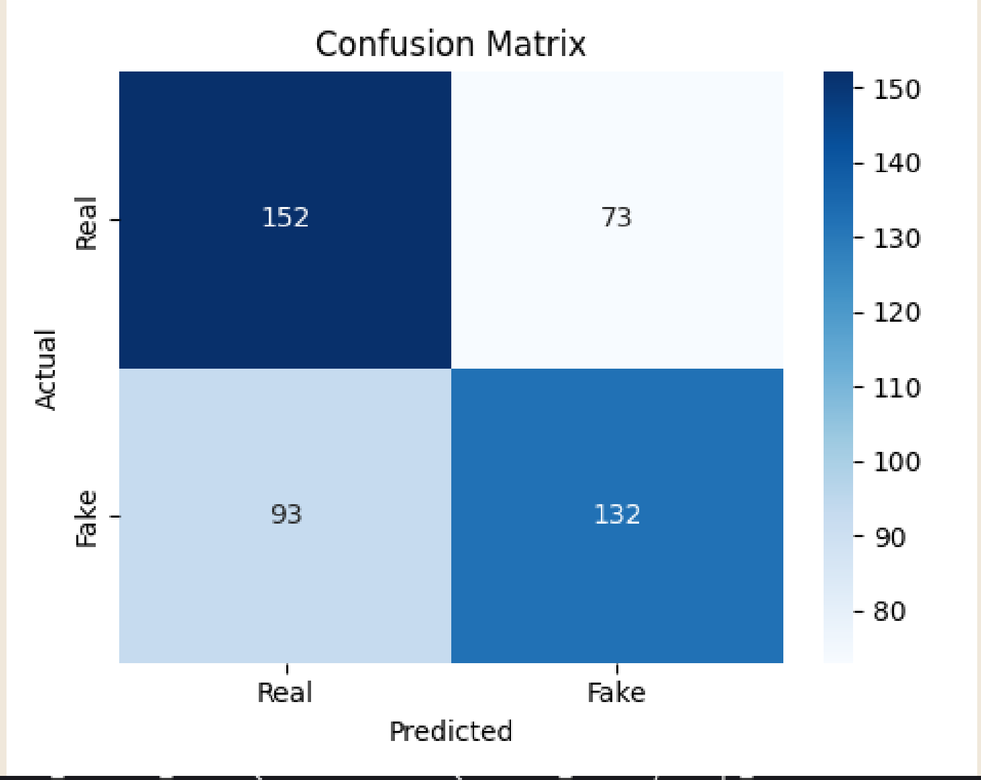
ReLU activation:
Introduces non-linearity
Prevents vanishing gradients
Efficient and stable for deep CNNs

224×224 → 112×112 → 56×56 → 28×28 → 7×7

BCEWithLogitsLoss safely turns the model's raw score into a probability inside the loss function, so training stays stable and accurate.

Results

```
Jan 8  4:33 AM
smitty@fedora:~/Downloads/ipdproject_e
~/Downloads/ipdproject_e
smitty@fedora:~/Downloads/ipdproject_exp — python train.py
smitty@fedora:~/Downloads/deepfake_detector_projec

GPU: NVIDIA GeForce RTX 3050 Laptop GPU
Epoch 1 [Train]: 100%|                                        | 525/525 [04:12<00:00,  2.08it/s]
Epoch 1 [Val]: 100%|                                          | 113/113 [00:51<00:00,  2.18it/s]

Epoch 1 | Train Loss: 0.6921 | Val Loss: 0.6849 | Val Acc: 0.6156
✓ Saved best model
Epoch 2 [Train]: 100%|                                        | 525/525 [04:15<00:00,  2.06it/s]
Epoch 2 [Val]: 100%|                                          | 113/113 [00:51<00:00,  2.17it/s]

Epoch 2 | Train Loss: 0.6827 | Val Loss: 0.6727 | Val Acc: 0.6556
✓ Saved best model
Epoch 3 [Train]: 100%|                                        | 525/525 [04:15<00:00,  2.05it/s]
Epoch 3 [Val]: 100%|                                          | 113/113 [00:51<00:00,  2.18it/s]

Epoch 3 | Train Loss: 0.6750 | Val Loss: 0.6649 | Val Acc: 0.6689
✓ Saved best model
Epoch 4 [Train]: 100%|                                        | 525/525 [04:15<00:00,  2.05it/s]
Epoch 4 [Val]: 100%|                                          | 113/113 [00:52<00:00,  2.17it/s]

Epoch 4 | Train Loss: 0.6685 | Val Loss: 0.6612 | Val Acc: 0.6422
✓ Saved best model
Epoch 5 [Train]: 100%|                                        | 525/525 [03:32<00:00,  2.47it/s]
Epoch 5 [Val]: 100%|                                          | 113/113 [00:43<00:00,  2.58it/s]

Epoch 5 | Train Loss: 0.6611 | Val Loss: 0.6510 | Val Acc: 0.6689
✓ Saved best model
Epoch 6 [Train]: 100%|                                        | 525/525 [03:36<00:00,  2.42it/s]
Epoch 6 [Val]: 100%|                                          | 113/113 [00:43<00:00,  2.58it/s]

Epoch 6 | Train Loss: 0.6578 | Val Loss: 0.6451 | Val Acc: 0.6822
✓ Saved best model
Epoch 7 [Train]: 100%|                                        | 525/525 [03:38<00:00,  2.40it/s]
Epoch 7 [Val]: 100%|                                          | 113/113 [00:44<00:00,  2.53it/s]

Epoch 7 | Train Loss: 0.6520 | Val Loss: 0.6410 | Val Acc: 0.6778
✓ Saved best model
Epoch 8 [Train]: 100%|                                        | 525/525 [03:36<00:00,  2.42it/s]
Epoch 8 [Val]: 100%|                                          | 113/113 [00:43<00:00,  2.58it/s]

Epoch 8 | Train Loss: 0.6471 | Val Loss: 0.6397 | Val Acc: 0.6556
✓ Saved best model
Epoch 9 [Train]: 100%|                                        | 525/525 [03:36<00:00,  2.43it/s]
Epoch 9 [Val]: 100%|                                          | 113/113 [00:43<00:00,  2.57it/s]

Epoch 9 | Train Loss: 0.6481 | Val Loss: 0.6406 | Val Acc: 0.6622
Epoch 10 [Train]: 100%|                                       | 525/525 [03:36<00:00,  2.42it/s]
Epoch 10 [Val]: 100%|                                         | 113/113 [00:43<00:00,  2.57it/s]
```

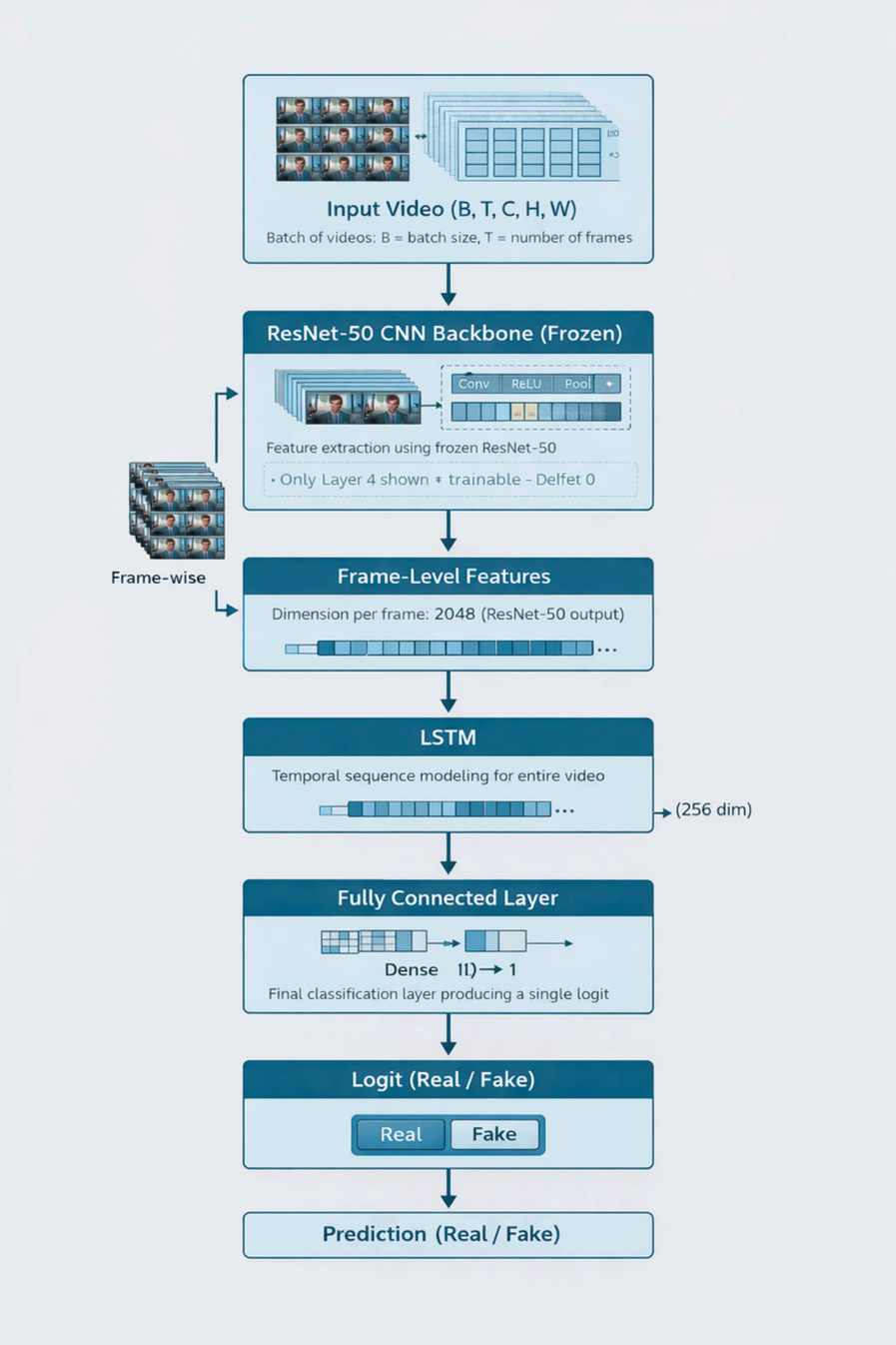



```
Evaluating: 100%|                                                    | 113/11

==================================================
TEST SET EVALUATION
==================================================
Accuracy       : 0.6311
Precision      : 0.6439
Recall         : 0.5867
F1-score       : 0.6140
==================================================
```

ONLY LSTM MODEL

The LSTM-only model processes a sequence of pre-extracted frame-level features to learn temporal consistency across a video. By modeling long-term dependencies using gated recurrent units, the LSTM detects unnatural motion patterns common in deepfakes. A final linear layer produces a video-level logit trained with BCEWithLogitsLoss for binary classification.

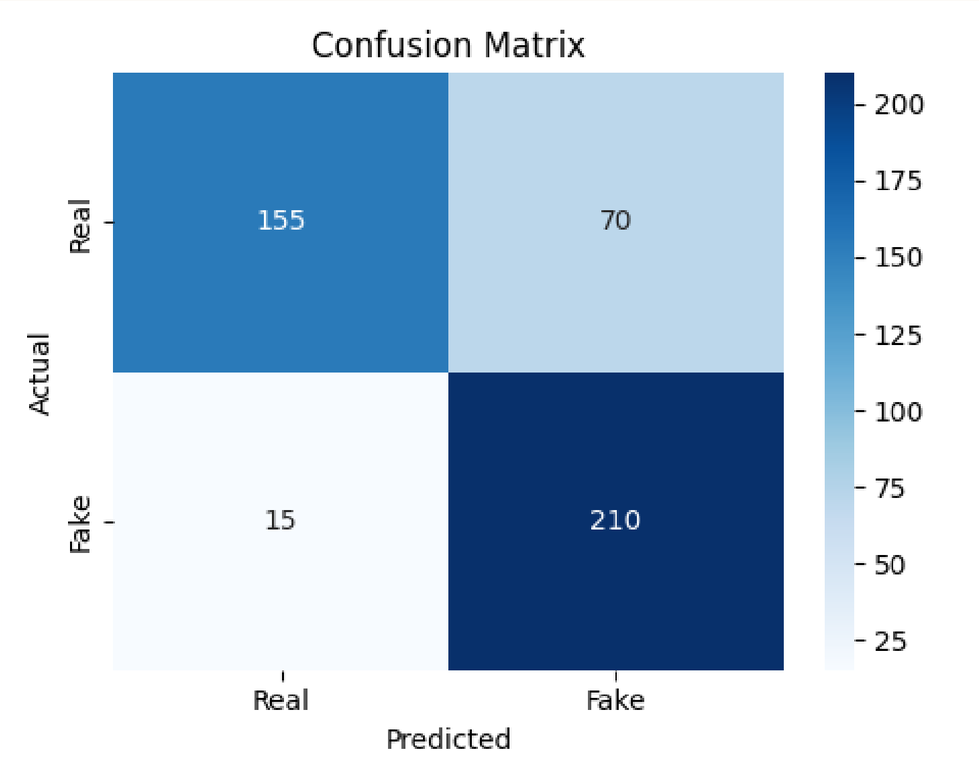
B = batch size
T = number of time steps (frames)
D = feature dimension per frame



Results

```
Using device: cuda
GPU: NVIDIA GeForce RTX 3050 Laptop GPU
Epoch 1 [Train]: 100%|                                                    | 525/525 [04:19<00:00,  2.02
Epoch 1 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.53

Epoch 1 | Train Loss: 0.6086 | Val Loss: 0.4796 | Val Acc: 0.8111
✓ Saved best model
Epoch 2 [Train]: 100%|                                                    | 525/525 [04:29<00:00,  1.95
Epoch 2 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.52

Epoch 2 | Train Loss: 0.4860 | Val Loss: 0.4593 | Val Acc: 0.8089
✓ Saved best model
Epoch 3 [Train]: 100%|                                                    | 525/525 [04:57<00:00,  1.76
Epoch 3 [Val]: 100%|                                                      | 113/113 [00:52<00:00,  2.13

Epoch 3 | Train Loss: 0.3880 | Val Loss: 0.3912 | Val Acc: 0.8289
✓ Saved best model
Epoch 4 [Train]: 100%|                                                    | 525/525 [05:15<00:00,  1.67
Epoch 4 [Val]: 100%|                                                      | 113/113 [00:52<00:00,  2.16
```

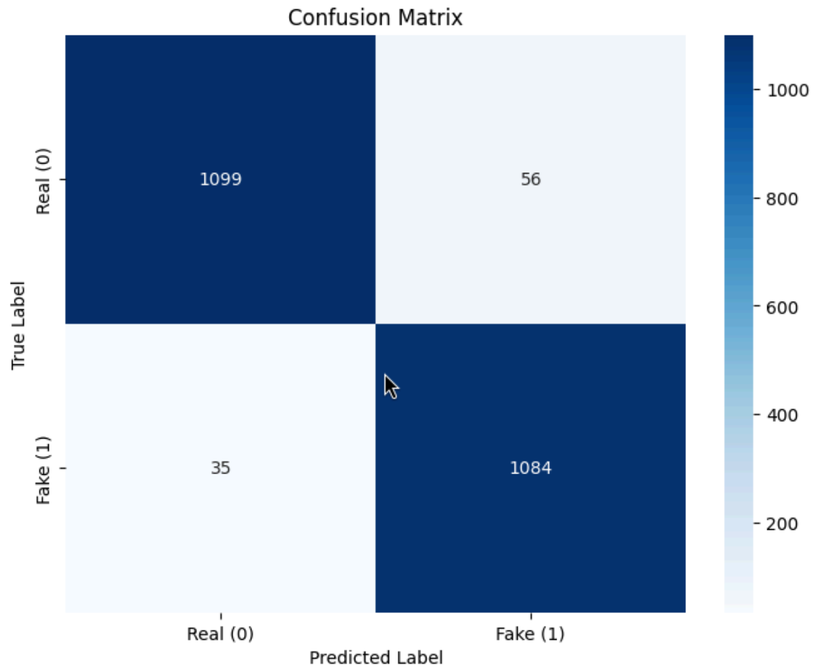
```
Evaluating: 100%|                                                         | 113/113 [20:26<00

Classification Report:

              precision    recall  f1-score   support

        Real       0.91      0.69      0.78       225
        Fake       0.75      0.93      0.83       225

    accuracy                           0.81       450
   macro avg       0.83      0.81      0.81       450
weighted avg       0.83      0.81      0.81       450
```



CNN+BILSTM ON SMALLER DATASET

```
Real video clips created: 5772
Fake video clips created: 5597
Total video clips: 11369
Getting all labels for stratification...
Finished getting labels.
Saving test set indices to 'test_indices.npy'...
This file will be loaded by the 03_evaluation notebook.

--- Data Split Summary ---
Total size: 11369
Training set size: 7276
Validation set size: 1819
Test set size (saved for later): 2274
Starting training...
Epoch 1/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 1/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 1 | Train Loss: 0.5034 | Val Loss: 0.3258 | Val Acc: 0.8725
Saving best model with accuracy: 0.8725
Epoch 2/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 2/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 2 | Train Loss: 0.3872 | Val Loss: 0.2745 | Val Acc: 0.8928
Saving best model with accuracy: 0.8928
Epoch 3/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 3/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 3 | Train Loss: 0.3225 | Val Loss: 0.2956 | Val Acc: 0.8873
Epoch 4/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 4/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 4 | Train Loss: 0.2839 | Val Loss: 0.2005 | Val Acc: 0.9329
Saving best model with accuracy: 0.9329
Epoch 5/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 5/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 5 | Train Loss: 0.2656 | Val Loss: 0.1935 | Val Acc: 0.9236
Epoch 6/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 6/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 6 | Train Loss: 0.2313 | Val Loss: 0.1464 | Val Acc: 0.9478
Saving best model with accuracy: 0.9478
Epoch 7/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 7/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 7 | Train Loss: 0.2202 | Val Loss: 0.2516 | Val Acc: 0.9164
Epoch 8/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 8/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 8 | Train Loss: 0.1920 | Val Loss: 0.1296 | Val Acc: 0.9610
Saving best model with accuracy: 0.9610
Epoch 9/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 9/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 9 | Train Loss: 0.1887 | Val Loss: 0.2708 | Val Acc: 0.9181
Epoch 10/10 Training:   0%|          | 0/910 [00:00<?, ?it/s]
Epoch 10/10 Validation:   0%|          | 0/228 [00:00<?, ?it/s]
Epoch 10 | Train Loss: 0.1728 | Val Loss: 0.1131 | Val Acc: 0.9544
Training complete!
```

```
--- Evaluation Metrics ---
Accuracy: 0.9600
Precision: 0.9509
Recall: 0.9687
F1 Score: 0.9597
ROC AUC: 0.9920

Generating Confusion Matrix...
```

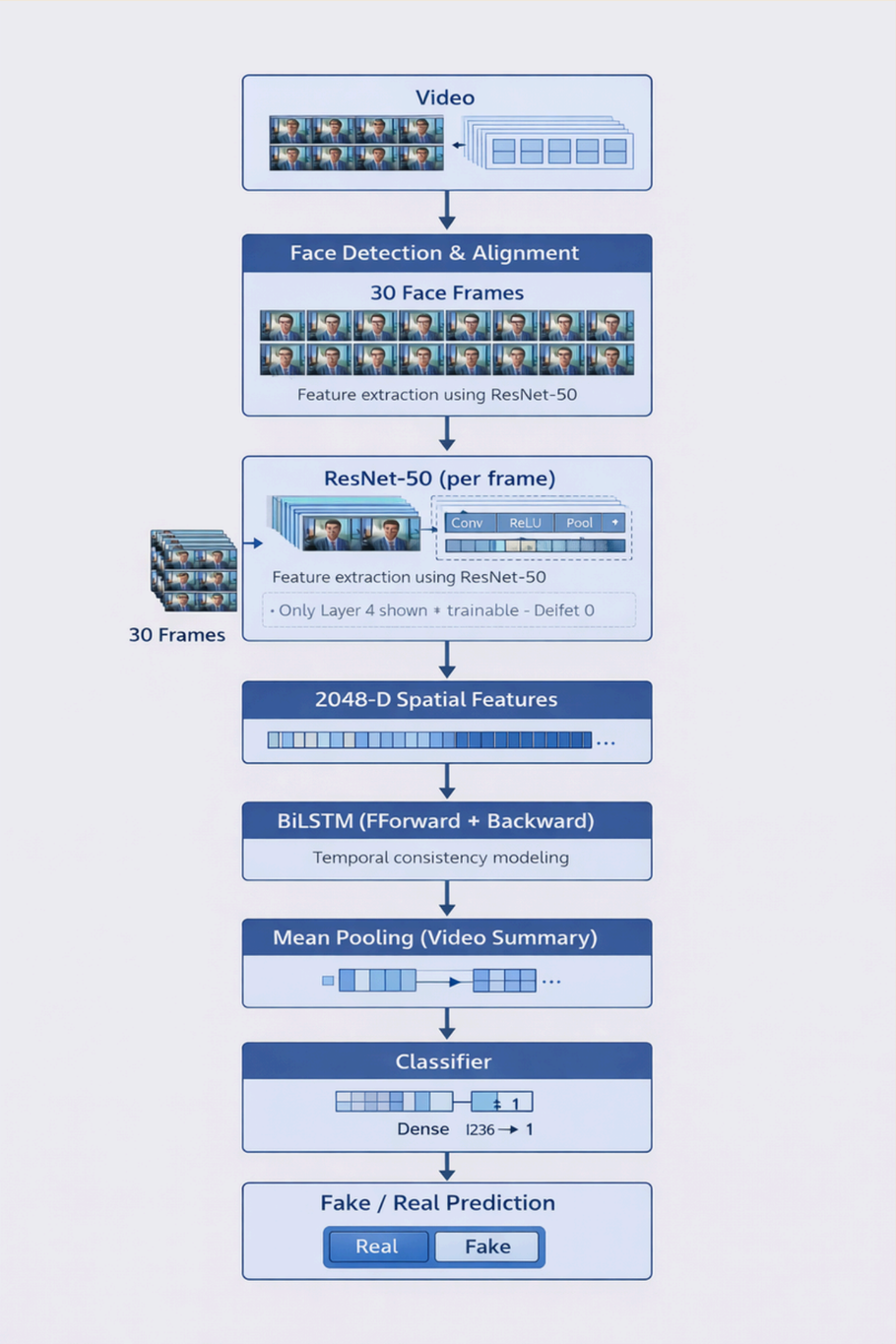


Overfitting

CNN+BILSTM MODEL

What the CNN does ResNet-50:
self.cnn = models.resnet50(...)
self.cnn.fc = nn.Identity()
Each frame is passed independently through the CNN.
For one frame:
Frame → ResNet → 2048-dim feature vector
For a full video:
30 frames → 30 feature vectors

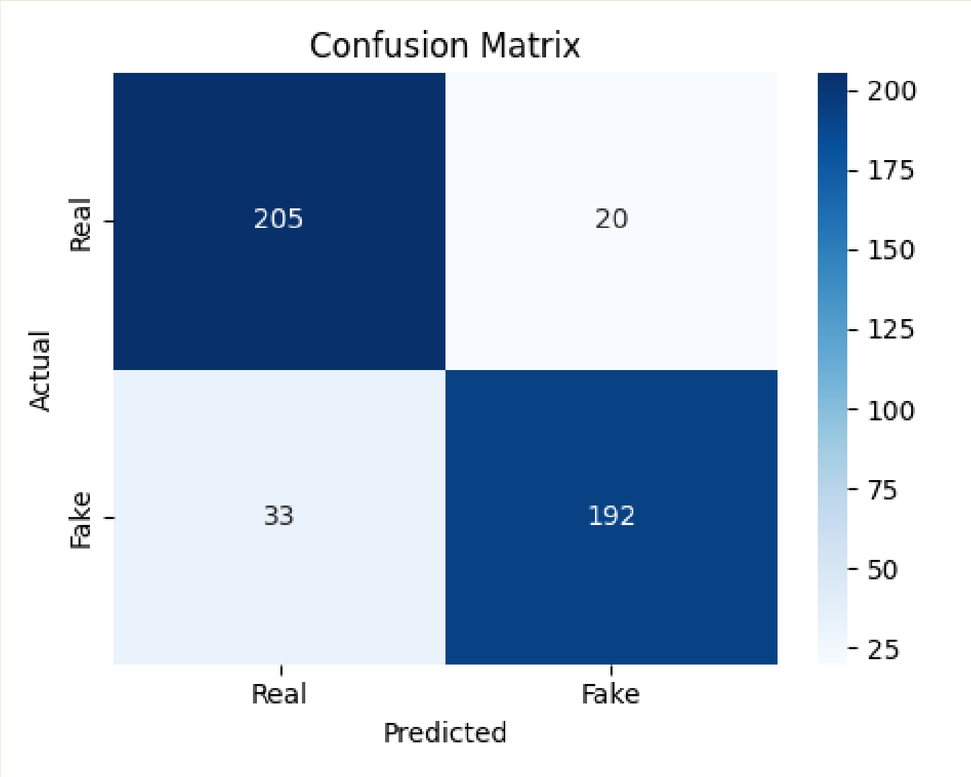
BiLSTM Stage – Temporal Consistency Learning
Why LSTM is needed
Deepfakes often fail over time, not in a single frame:
Eye blinking frequency
Lip sync timing
Micro-expressions
Frame-to-frame flickering
CNN alone cannot detect these.
The BiLSTM processes the feature sequence forward and backward.

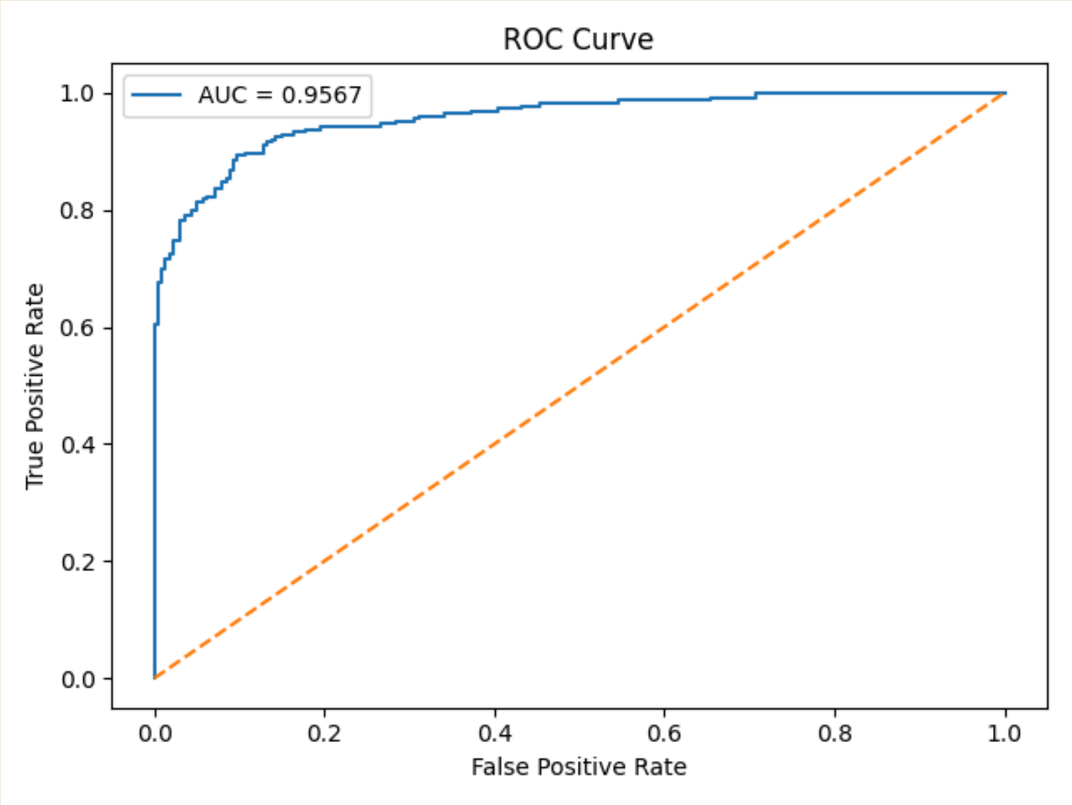


```
GPU: NVIDIA GeForce RTX 3050 Laptop GPU
Epoch 1 [Train]: 100%|                                                    | 525/525 [04:40<00:00,  1.87it/s]
Epoch 1 [Val]: 100%|                                                      | 113/113 [00:46<00:00,  2.43it/s]

Epoch 1 | Train Loss: 0.5697 | Val Loss: 0.3961 | Val Acc: 0.8467
✓ Saved best model
Epoch 2 [Train]: 100%|                                                    | 525/525 [04:44<00:00,  1.84it/s]
Epoch 2 [Val]: 100%|                                                      | 113/113 [00:48<00:00,  2.35it/s]

Epoch 2 | Train Loss: 0.4639 | Val Loss: 0.3533 | Val Acc: 0.8444
✓ Saved best model
Epoch 3 [Train]: 100%|                                                    | 525/525 [04:32<00:00,  1.93it/s]
Epoch 3 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.53it/s]

Epoch 3 | Train Loss: 0.3842 | Val Loss: 0.3415 | Val Acc: 0.8422
✓ Saved best model
Epoch 4 [Train]: 100%|                                                    | 525/525 [04:30<00:00,  1.94it/s]
Epoch 4 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.52it/s]

Epoch 4 | Train Loss: 0.3380 | Val Loss: 0.3831 | Val Acc: 0.8444
Epoch 5 [Train]: 100%|                                                    | 525/525 [04:29<00:00,  1.95it/s]
Epoch 5 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.53it/s]

Epoch 5 | Train Loss: 0.2563 | Val Loss: 0.3167 | Val Acc: 0.8644
✓ Saved best model
Epoch 6 [Train]: 100%|                                                    | 525/525 [04:30<00:00,  1.94it/s]
Epoch 6 [Val]: 100%|                                                      | 113/113 [00:44<00:00,  2.52it/s]

Epoch 6 | Train Loss: 0.2092 | Val Loss: 0.4862 | Val Acc: 0.8200
Epoch 7 [Train]: 100%|                                                    | 525/525 [04:30<00:00,  1.94it/s]
Epoch 7 [Val]: 100%|                                                      | 113/113 [00:45<00:00,  2.51it/s]

Epoch 7 | Train Loss: 0.1838 | Val Loss: 0.4442 | Val Acc: 0.8378
Epoch 8 [Train]: 100%|                                                    | 525/525 [04:30<00:00,  1.94it/s]
```

```
  model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
Evaluating: 100%|                                                         | 113/113 [00:53<00:00,  2.13it/s]

Classification Report:

              precision    recall  f1-score   support

        Real       0.91      0.80      0.85       225
        Fake       0.82      0.92      0.87       225

    accuracy                           0.86       450
   macro avg       0.87      0.86      0.86       450
weighted avg       0.87      0.86      0.86       450


✓ All plots saved successfully
(ipd) smitty@fedora:~/Downloads/ipdproject$
```

```
GPU: NVIDIA GeForce RTX 3050 Laptop GPU
Test samples: 450
/home/smitty/Downloads/ipdproject/evaluate_test.py:57: FutureWarning: You are using `torch.load` with `we
ights_only=False` (the current default value), which uses the default pickle module implicitly. It is pos
sible to construct malicious pickle data which will execute arbitrary code during unpickling (See https:/
/github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release
, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be
 executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unles
s they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend yo
u start setting `weights_only=True` for any use case where you don't have full control of the loaded file
. Please open an issue on GitHub for any issues related to this experimental feature.
  model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
Evaluating: 100%|                                                         | 113/113 [00:42<00:00,  2.68it/s]

==================================================
TEST SET EVALUATION
==================================================
Accuracy      : 0.8600
Precision     : 0.8214
Recall        : 0.9200
F1-score      : 0.8679
==================================================
```





If you randomly pick one fake video and one real video, model will assign a higher fake probability to the fake video ~95.7% of the time.